SQL Interview Questions & Answers Cheat Sheet

Table of Contents

SQL Fundamentals

Q1: What is SQL and what are its main components?

Answer:
SQL (Structured Query Language) is a standardized language for managing relational databases.

Main SQL Sublanguages:

- **DDL (Data Definition Language)**: CREATE, ALTER, DROP, TRUNCATE
- **DML (Data Manipulation Language)**: INSERT, UPDATE, DELETE, MERGE
- **DQL (Data Query Language)**: SELECT
- **DCL (Data Control Language)**: GRANT, REVOKE
- **TCL (Transaction Control Language)**: COMMIT, ROLLBACK, SAVEPOINT

Popular SQL Databases:

- MySQL, PostgreSQL, SQL Server, Oracle, SQLite

Q2: What is the difference between SQL and NoSQL databases?

Answer:

Aspect	SQL (Relational)	NoSQL (Non-Relational)
Schema	Fixed schema, predefined structure	Flexible schema, dynamic structure
Scalability	Vertical scaling (scale up)	Horizontal scaling (scale out)
ACID	Full ACID compliance	Eventual consistency
Query Language	Standardized SQL	Varies by database
Use Cases	Complex queries, transactions	Big data, real-time web apps
Examples	MySQL, PostgreSQL, Oracle	MongoDB, Cassandra, Redis

When to Use SQL:

- Complex relationships between data
- ACID transactions required
- Well-defined schema
- Complex queries and reporting

When to Use NoSQL:

- Rapid development with changing requirements
- Large scale and high performance
- Simple queries
- Distributed architecture

Data Types & Constraints

Q3: What are the common SQL data types?

Answer:

Numeric Types:

- **INT/INTEGER**: Whole numbers (-2,147,483,648 to 2,147,483,647)
- **BIGINT**: Large integers
- **DECIMAL(p,s)**: Fixed-point numbers
- **FLOAT/REAL**: Floating-point numbers

String Types:

- **VARCHAR(n)**: Variable-length strings (up to n characters)
- **CHAR(n)**: Fixed-length strings

- **TEXT**: Large text data

Date/Time Types:

- **DATE**: Date values (YYYY-MM-DD)
- **TIME**: Time values (HH:MM:SS)
- **DATETIME/TIMESTAMP**: Date and time combined

Boolean:

- **BOOLEAN**: TRUE/FALSE values

Q4: Explain PRIMARY KEY, FOREIGN KEY, and other constraints.

Answer:

PRIMARY KEY:

- Uniquely identifies each row in a table
- Cannot be NULL
- Only one per table
- Automatically creates unique index

FOREIGN KEY:

- References PRIMARY KEY of another table
- Enforces referential integrity
- Can be NULL (unless specified otherwise)
- Multiple foreign keys allowed

Other Constraints:

- **UNIQUE**: Ensures all values in column are distinct
- **NOT NULL**: Column cannot contain NULL values
- **CHECK**: Validates data based on condition
- **DEFAULT**: Provides default value when none specified

Example:

```
CREATE TABLE Orders (
    OrderID INT PRIMARY KEY,
    CustomerID INT NOT NULL,
    OrderDate DATE DEFAULT CURRENT_DATE,
    TotalAmount DECIMAL(10,2) CHECK (TotalAmount &gt; 0),
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
);
```

Basic Queries & Filtering

Q5: What is the difference between WHERE and HAVING?

Answer:

Aspect	WHERE	HAVING
Purpose	Filters individual rows	Filters groups after GROUP BY
Usage	Before GROUP BY	After GROUP BY
Aggregate Functions	Cannot use directly	Can use aggregate functions
Performance	Faster (filters before grouping)	Slower (filters after grouping)

Example:

```
-- WHERE: Filter individual rows
SELECT CustomerID, COUNT(*) as OrderCount
FROM Orders
WHERE OrderDate &gt;= '2024-01-01'  -- Filter rows first
GROUP BY CustomerID
HAVING COUNT(*) &gt; 5;  -- Then filter groups

-- Incorrect usage:
SELECT CustomerID, COUNT(*) as OrderCount
FROM Orders
WHERE COUNT(*) &gt; 5  -- ERROR: Cannot use aggregate in WHERE
GROUP BY CustomerID;
```

Q6: Explain NULL values and how to handle them.

Answer:

NULL Characteristics:

- Represents missing or unknown data
- NULL ≠ 0 or empty string
- Any operation with NULL returns NULL
- Cannot use = or != with NULL

Handling NULL Values:

```
-- Check for NULL
SELECT * FROM Employees WHERE ManagerID IS NULL;
SELECT * FROM Employees WHERE ManagerID IS NOT NULL;

-- NULL in calculations
SELECT Salary + Bonus FROM Employees;  -- Returns NULL if Bonus is NULL

-- Handle NULL with functions
```

```
SELECT
    COALESCE(Bonus, 0) as BonusAmount,  -- Return 0 if Bonus is NULL
    ISNULL(Bonus, 0) as BonusAmount,    -- SQL Server syntax
    IFNULL(Bonus, 0) as BonusAmount     -- MySQL syntax
FROM Employees;

-- NULL in aggregations
SELECT AVG(Salary) FROM Employees;  -- Ignores NULL values
SELECT COUNT(Bonus) FROM Employees; -- Counts non-NULL values only
```

Joins & Relationships

Q7: Explain different types of JOINs with examples.

Answer:

INNER JOIN:
Returns only matching records from both tables.

```
SELECT c.CustomerName, o.OrderDate
FROM Customers c
INNER JOIN Orders o ON c.CustomerID = o.CustomerID;
```

LEFT JOIN (LEFT OUTER JOIN):
Returns all records from left table, matched records from right table.

```
SELECT c.CustomerName, o.OrderDate
FROM Customers c
LEFT JOIN Orders o ON c.CustomerID = o.CustomerID;
-- Shows all customers, even those without orders
```

RIGHT JOIN (RIGHT OUTER JOIN):
Returns all records from right table, matched records from left table.

```
SELECT c.CustomerName, o.OrderDate
FROM Customers c
RIGHT JOIN Orders o ON c.CustomerID = o.CustomerID;
-- Shows all orders, even those without customer details
```

FULL OUTER JOIN:
Returns all records when there's a match in either table.

```
SELECT c.CustomerName, o.OrderDate
FROM Customers c
FULL OUTER JOIN Orders o ON c.CustomerID = o.CustomerID;
-- Shows all customers and all orders
```

CROSS JOIN:
Cartesian product of both tables.

```
SELECT c.CustomerName, p.ProductName
FROM Customers c
CROSS JOIN Products p;
-- Every customer paired with every product
```

SELF JOIN:
Table joined with itself.

```
SELECT e1.EmployeeName as Employee, e2.EmployeeName as Manager
FROM Employees e1
LEFT JOIN Employees e2 ON e1.ManagerID = e2.EmployeeID;
```

Q8: What is the difference between UNION and UNION ALL?

Answer:

Aspect	UNION	UNION ALL
Duplicates	Removes duplicates	Keeps all duplicates
Performance	Slower (duplicate removal)	Faster (no duplicate check)
Sorting	Implicitly sorts results	No implicit sorting

Example:

```
-- UNION - removes duplicates
SELECT City FROM Customers
UNION
SELECT City FROM Suppliers;

-- UNION ALL - keeps duplicates
SELECT City FROM Customers
UNION ALL
SELECT City FROM Suppliers;

-- Requirements for UNION:
-- 1. Same number of columns
-- 2. Compatible data types
-- 3. Same column order
```

Aggregate Functions & GROUP BY

Q9: Explain aggregate functions and their usage.

Answer:

Common Aggregate Functions:

- **COUNT()**: Number of rows
- **SUM()**: Total of numeric values
- **AVG()**: Average of numeric values
- **MIN()**: Minimum value
- **MAX()**: Maximum value

Examples:

```
-- Basic aggregations
SELECT
    COUNT(*) as TotalOrders,
    COUNT(CustomerID) as OrdersWithCustomers,  -- Excludes NULLs
    SUM(TotalAmount) as TotalRevenue,
    AVG(TotalAmount) as AverageOrderValue,
    MIN(OrderDate) as FirstOrder,
    MAX(OrderDate) as LastOrder
FROM Orders;

-- GROUP BY with aggregations
SELECT
    CustomerID,
    COUNT(*) as OrderCount,
    SUM(TotalAmount) as TotalSpent,
    AVG(TotalAmount) as AvgOrderValue
FROM Orders
GROUP BY CustomerID
HAVING COUNT(*) &gt; 5  -- Customers with more than 5 orders
ORDER BY TotalSpent DESC;
```

Q10: How do you handle complex grouping scenarios?

Answer:

Multiple Column Grouping:

```
SELECT
    YEAR(OrderDate) as OrderYear,
    MONTH(OrderDate) as OrderMonth,
    COUNT(*) as OrderCount,
    SUM(TotalAmount) as MonthlyRevenue
FROM Orders
```

```
GROUP BY YEAR(OrderDate), MONTH(OrderDate)
ORDER BY OrderYear, OrderMonth;
```

ROLLUP and CUBE (Advanced Grouping):

```
-- ROLLUP: Hierarchical subtotals
SELECT
    Region,
    Country,
    SUM(Sales) as TotalSales
FROM SalesData
GROUP BY ROLLUP(Region, Country);

-- CUBE: All possible combinations
SELECT
    Region,
    Country,
    Product,
    SUM(Sales) as TotalSales
FROM SalesData
GROUP BY CUBE(Region, Country, Product);
```

Subqueries & CTEs

Q11: What are subqueries and when should you use them?

Answer:

Subquery Types:

- **Scalar Subquery**: Returns single value
- **Row Subquery**: Returns single row, multiple columns
- **Column Subquery**: Returns multiple rows, single column
- **Table Subquery**: Returns multiple rows and columns

Examples:

```
-- Scalar subquery
SELECT *
FROM Products
WHERE Price &gt; (SELECT AVG(Price) FROM Products);

-- Correlated subquery
SELECT CustomerID, CustomerName
FROM Customers c
WHERE EXISTS (
    SELECT 1
    FROM Orders o
    WHERE o.CustomerID = c.CustomerID
    AND o.OrderDate &gt; '2024-01-01'
```

```
);

-- Subquery with IN
SELECT *
FROM Employees
WHERE DepartmentID IN (
    SELECT DepartmentID
    FROM Departments
    WHERE Location = 'New York'
);
```

Q12: What are CTEs and how do they differ from subqueries?

Answer:

Common Table Expression (CTE):
Temporary named result set that exists within scope of single statement.

Advantages over Subqueries:

- **Reusability**: Can reference multiple times
- **Readability**: Easier to understand complex logic
- **Recursive Capabilities**: Support recursive queries
- **Performance**: Can be more efficient for complex queries

Basic CTE:

```
WITH HighValueCustomers AS (
    SELECT CustomerID, SUM(TotalAmount) as TotalSpent
    FROM Orders
    GROUP BY CustomerID
    HAVING SUM(TotalAmount) &gt; 10000
)
SELECT c.CustomerName, hvc.TotalSpent
FROM Customers c
JOIN HighValueCustomers hvc ON c.CustomerID = hvc.CustomerID;
```

Multiple CTEs:

```
WITH
MonthlyOrders AS (
    SELECT
        YEAR(OrderDate) as OrderYear,
        MONTH(OrderDate) as OrderMonth,
        COUNT(*) as OrderCount
    FROM Orders
    GROUP BY YEAR(OrderDate), MONTH(OrderDate)
),
AvgMonthlyOrders AS (
    SELECT AVG(OrderCount) as AvgOrders
    FROM MonthlyOrders
```

```
)
SELECT mo.*
FROM MonthlyOrders mo, AvgMonthlyOrders amo
WHERE mo.OrderCount &gt; amo.AvgOrders;
```

Recursive CTE:

```
-- Find employee hierarchy
WITH EmployeeHierarchy AS (
    -- Anchor member: top-level managers
    SELECT EmployeeID, EmployeeName, ManagerID, 1 as Level
    FROM Employees
    WHERE ManagerID IS NULL

    UNION ALL

    -- Recursive member: employees with managers
    SELECT e.EmployeeID, e.EmployeeName, e.ManagerID, eh.Level + 1
    FROM Employees e
    JOIN EmployeeHierarchy eh ON e.ManagerID = eh.EmployeeID
)
SELECT * FROM EmployeeHierarchy
ORDER BY Level, EmployeeName;
```

Window Functions

Q13: Explain window functions and their syntax.

Answer:

Window Function Syntax:

```
function_name([arguments]) OVER (
    [PARTITION BY column1, column2, ...]
    [ORDER BY column1 [ASC|DESC], column2 [ASC|DESC], ...]
    [ROWS/RANGE frame_specification]
)
```

Components:

- **PARTITION BY**: Divides result set into partitions
- **ORDER BY**: Defines order within each partition
- **Frame**: Defines subset of partition for calculation

Q14: What are the different types of window functions?

Answer:

1. Ranking Functions:

```
SELECT
    EmployeeName,
    Salary,
    ROW_NUMBER() OVER (ORDER BY Salary DESC) as RowNum,
    RANK() OVER (ORDER BY Salary DESC) as Rank,
    DENSE_RANK() OVER (ORDER BY Salary DESC) as DenseRank,
    NTILE(4) OVER (ORDER BY Salary DESC) as Quartile
FROM Employees;
```

2. Aggregate Window Functions:

```
SELECT
    OrderDate,
    TotalAmount,
    SUM(TotalAmount) OVER (ORDER BY OrderDate) as RunningTotal,
    AVG(TotalAmount) OVER (ORDER BY OrderDate ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) a
FROM Orders;
```

3. Value Functions:

```
SELECT
    EmployeeName,
    Salary,
    LAG(Salary, 1) OVER (ORDER BY Salary) as PrevSalary,
    LEAD(Salary, 1) OVER (ORDER BY Salary) as NextSalary,
    FIRST_VALUE(Salary) OVER (ORDER BY Salary) as MinSalary,
    LAST_VALUE(Salary) OVER (ORDER BY Salary ROWS UNBOUNDED FOLLOWING) as MaxSalary
FROM Employees;
```

4. Statistical Functions:

```
SELECT
    ProductID,
    Price,
    PERCENT_RANK() OVER (ORDER BY Price) as PercentRank,
    CUME_DIST() OVER (ORDER BY Price) as CumulativeDistribution
FROM Products;
```

Advanced SQL Concepts

Q15: What is the difference between RANK(), DENSE_RANK(), and ROW_NUMBER()?

Answer:

Function	Gaps in Ranking	Duplicate Handling	Use Case
ROW_NUMBER()	No gaps	Unique numbers for duplicates	Unique identifier needed
RANK()	Creates gaps	Same rank for duplicates	Traditional ranking
DENSE_RANK()	No gaps	Same rank for duplicates	Continuous ranking

Example:

```
-- Sample data: Salaries: 100000, 90000, 90000, 80000
SELECT
    EmployeeName,
    Salary,
    ROW_NUMBER() OVER (ORDER BY Salary DESC) as RowNum,      -- 1,2,3,4
    RANK() OVER (ORDER BY Salary DESC) as Rank,             -- 1,2,2,4
    DENSE_RANK() OVER (ORDER BY Salary DESC) as DenseRank   -- 1,2,2,3
FROM Employees;
```

Q16: How do you find duplicate records in a table?

Answer:

Method 1: Using GROUP BY and HAVING

```
-- Find duplicate email addresses
SELECT Email, COUNT(*) as DuplicateCount
FROM Users
GROUP BY Email
HAVING COUNT(*) &gt; 1;

-- Find complete duplicate records
SELECT FirstName, LastName, Email, COUNT(*) as DuplicateCount
FROM Users
GROUP BY FirstName, LastName, Email
HAVING COUNT(*) &gt; 1;
```

Method 2: Using Window Functions

```
-- Identify duplicates with row numbers
WITH DuplicateRecords AS (
    SELECT *,
           ROW_NUMBER() OVER (PARTITION BY Email ORDER BY UserID) as RowNum
    FROM Users
```

```
)
SELECT * FROM DuplicateRecords WHERE RowNum &gt; 1;
```

Method 3: Self Join

```
-- Find duplicate emails using self join
SELECT DISTINCT u1.*
FROM Users u1
JOIN Users u2 ON u1.Email = u2.Email AND u1.UserID != u2.UserID;
```

Removing Duplicates:

```
-- Delete duplicates keeping the first occurrence
DELETE FROM Users
WHERE UserID NOT IN (
    SELECT MIN(UserID)
    FROM Users
    GROUP BY Email
);
```

Performance & Optimization

Q17: How do you optimize slow SQL queries?

Answer:

Query Optimization Techniques:

1. Use Indexes Effectively:

```
-- Create indexes on frequently queried columns
CREATE INDEX idx_customer_email ON Customers(Email);
CREATE INDEX idx_order_date ON Orders(OrderDate);

-- Composite indexes for multi-column queries
CREATE INDEX idx_order_customer_date ON Orders(CustomerID, OrderDate);
```

**2. Avoid SELECT :*

```
-- Bad
SELECT * FROM Orders WHERE OrderDate &gt; '2024-01-01';

-- Good
SELECT OrderID, CustomerID, TotalAmount
FROM Orders
WHERE OrderDate &gt; '2024-01-01';
```

3. Use LIMIT for Large Results:

```
-- Limit results when you don't need all rows
SELECT * FROM Products ORDER BY Price DESC LIMIT 10;
```

4. Optimize WHERE Clauses:

```
-- Use indexes effectively
WHERE CustomerID = 123  -- Good for indexed column

-- Avoid functions in WHERE clause
WHERE YEAR(OrderDate) = 2024  -- Bad
WHERE OrderDate &gt;= '2024-01-01' AND OrderDate &lt; '2025-01-01'  -- Good
```

5. Use EXISTS instead of IN for Subqueries:

```
-- Less efficient
SELECT * FROM Customers
WHERE CustomerID IN (SELECT CustomerID FROM Orders);

-- More efficient
SELECT * FROM Customers c
WHERE EXISTS (SELECT 1 FROM Orders o WHERE o.CustomerID = c.CustomerID);
```

Q18: Explain indexes and their types.

Answer:

Index Types:

1. Clustered Index:

- Physical ordering of data
- One per table
- Usually on primary key

2. Non-Clustered Index:

- Logical ordering with pointers to data
- Multiple per table
- Faster for specific queries

3. Unique Index:

- Ensures uniqueness
- Automatically created for primary key and unique constraints

4. Composite Index:

- Spans multiple columns
- Order of columns matters

Index Management:

```
-- Create index
CREATE INDEX idx_customer_lastname ON Customers(LastName);

-- Create composite index
CREATE INDEX idx_order_customer_date ON Orders(CustomerID, OrderDate);

-- Drop index
DROP INDEX idx_customer_lastname;

-- View index usage (SQL Server)
SELECT * FROM sys.dm_db_index_usage_stats;
```

When to Use Indexes:

- **Create**: Frequently queried columns, WHERE clauses, JOIN conditions
- **Avoid**: Small tables, frequently updated columns, tables with high insert/update volume

Database Design & Normalization

Q19: Explain database normalization and its forms.

Answer:

Normalization: Process of organizing data to minimize redundancy and dependency.

Normal Forms:

1st Normal Form (1NF):

- Eliminate repeating groups
- Each cell contains single value
- Each record is unique

Before 1NF:

CustomerName	Phone
John Smith	123-4567, 987-6543

After 1NF:

CustomerName	Phone
John Smith	123-4567
John Smith	987-6543

2nd Normal Form (2NF):

- Must be in 1NF
- Eliminate partial dependencies
- Non-key attributes depend on entire primary key

3rd Normal Form (3NF):

- Must be in 2NF
- Eliminate transitive dependencies
- Non-key attributes depend only on primary key

Denormalization:
Sometimes acceptable for performance reasons in data warehouses.

Q20: How do you design a database schema?

Answer:

Database Design Steps:

1. Requirements Analysis:

- Understand business rules
- Identify entities and relationships
- Define data requirements

2. Conceptual Design:

- Create Entity-Relationship (ER) diagram
- Define entities, attributes, and relationships

3. Logical Design:

- Convert ER diagram to relational model
- Define tables, columns, and relationships
- Apply normalization rules

4. Physical Design:

- Choose data types
- Define indexes
- Consider partitioning

Example E-commerce Schema:

```
-- Customers table
CREATE TABLE Customers (
    CustomerID INT PRIMARY KEY,
    FirstName VARCHAR(50) NOT NULL,
    LastName VARCHAR(50) NOT NULL,
    Email VARCHAR(100) UNIQUE NOT NULL,
```

```
    Phone VARCHAR(20),
    CreatedDate DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- Orders table
CREATE TABLE Orders (
    OrderID INT PRIMARY KEY,
    CustomerID INT NOT NULL,
    OrderDate DATETIME DEFAULT CURRENT_TIMESTAMP,
    TotalAmount DECIMAL(10,2) NOT NULL,
    Status VARCHAR(20) DEFAULT 'Pending',
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
);

-- Order Items table
CREATE TABLE OrderItems (
    OrderItemID INT PRIMARY KEY,
    OrderID INT NOT NULL,
    ProductID INT NOT NULL,
    Quantity INT NOT NULL,
    UnitPrice DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (OrderID) REFERENCES Orders(OrderID),
    FOREIGN KEY (ProductID) REFERENCES Products(ProductID)
);
```

Data Manipulation (DML)

Q21: Explain INSERT, UPDATE, and DELETE operations.

Answer:

INSERT Operations:

```
-- Insert single record
INSERT INTO Customers (FirstName, LastName, Email)
VALUES ('John', 'Doe', 'john.doe@email.com');

-- Insert multiple records
INSERT INTO Customers (FirstName, LastName, Email) VALUES
('Jane', 'Smith', 'jane.smith@email.com'),
('Bob', 'Johnson', 'bob.johnson@email.com');

-- Insert from another table
INSERT INTO ArchivedOrders (OrderID, CustomerID, OrderDate, TotalAmount)
SELECT OrderID, CustomerID, OrderDate, TotalAmount
FROM Orders
WHERE OrderDate &lt; '2023-01-01';

-- Insert with subquery
INSERT INTO TopCustomers (CustomerID, TotalSpent)
SELECT CustomerID, SUM(TotalAmount)
FROM Orders
```

```
GROUP BY CustomerID
HAVING SUM(TotalAmount) &gt; 10000;
```

UPDATE Operations:

```
-- Update single record
UPDATE Customers
SET Email = 'newemail@example.com'
WHERE CustomerID = 1;

-- Update multiple columns
UPDATE Products
SET Price = Price * 1.1,  -- Increase by 10%
    LastUpdated = CURRENT_TIMESTAMP
WHERE CategoryID = 1;

-- Update with JOIN
UPDATE o
SET o.Status = 'Shipped'
FROM Orders o
JOIN Customers c ON o.CustomerID = c.CustomerID
WHERE c.CustomerType = 'Premium' AND o.OrderDate &gt;= '2024-01-01';

-- Update with subquery
UPDATE Employees
SET Salary = Salary * 1.05
WHERE DepartmentID IN (
    SELECT DepartmentID
    FROM Departments
    WHERE Performance = 'Excellent'
);
```

DELETE Operations:

```
-- Delete specific records
DELETE FROM Orders WHERE Status = 'Cancelled';

-- Delete with JOIN
DELETE o
FROM Orders o
JOIN Customers c ON o.CustomerID = c.CustomerID
WHERE c.Status = 'Inactive';

-- Delete duplicates
DELETE FROM Customers
WHERE CustomerID NOT IN (
    SELECT MIN(CustomerID)
    FROM Customers
    GROUP BY Email
);
```

Q22: What is MERGE statement and when to use it?

Answer:

MERGE Statement: Combines INSERT, UPDATE, and DELETE operations in single statement based on matching conditions.

Syntax:

```
MERGE target_table AS target
USING source_table AS source
ON merge_condition
WHEN MATCHED THEN update_statement
WHEN NOT MATCHED BY TARGET THEN insert_statement
WHEN NOT MATCHED BY SOURCE THEN delete_statement;
```

Example - Synchronizing Customer Data:

```
MERGE Customers AS target
USING CustomerUpdates AS source
ON target.CustomerID = source.CustomerID
WHEN MATCHED AND target.Email != source.Email THEN
    UPDATE SET
        Email = source.Email,
        LastUpdated = CURRENT_TIMESTAMP
WHEN NOT MATCHED BY TARGET THEN
    INSERT (CustomerID, FirstName, LastName, Email)
    VALUES (source.CustomerID, source.FirstName, source.LastName, source.Email)
WHEN NOT MATCHED BY SOURCE THEN
    DELETE;
```

Use Cases:

- Data synchronization
- Upsert operations (Update or Insert)
- Data warehouse loading
- Maintaining lookup tables

Practical Coding Problems

Q23: Find the second highest salary in each department.

Answer:

Method 1: Using Window Functions

```
WITH RankedSalaries AS (
    SELECT
```

```
        EmployeeID,
        EmployeeName,
        DepartmentID,
        Salary,
        DENSE_RANK() OVER (PARTITION BY DepartmentID ORDER BY Salary DESC) as SalaryRank
    FROM Employees
)
SELECT * FROM RankedSalaries WHERE SalaryRank = 2;
```

Method 2: Using Subquery

```
SELECT e1.*
FROM Employees e1
WHERE 2 = (
    SELECT COUNT(DISTINCT e2.Salary)
    FROM Employees e2
    WHERE e2.DepartmentID = e1.DepartmentID
    AND e2.Salary &gt;= e1.Salary
);
```

Q24: Find customers who have not placed any orders.

Answer:

Method 1: Using LEFT JOIN

```
SELECT c.*
FROM Customers c
LEFT JOIN Orders o ON c.CustomerID = o.CustomerID
WHERE o.CustomerID IS NULL;
```

Method 2: Using NOT EXISTS

```
SELECT c.*
FROM Customers c
WHERE NOT EXISTS (
    SELECT 1
    FROM Orders o
    WHERE o.CustomerID = c.CustomerID
);
```

Method 3: Using NOT IN (be careful with NULLs)

```
SELECT c.*
FROM Customers c
WHERE c.CustomerID NOT IN (
    SELECT DISTINCT CustomerID
    FROM Orders
    WHERE CustomerID IS NOT NULL
);
```

Q25: Calculate running total and moving average.

Answer:

```
SELECT
    OrderDate,
    TotalAmount,
    -- Running total
    SUM(TotalAmount) OVER (ORDER BY OrderDate) as RunningTotal,

    -- Moving average (last 3 orders)
    AVG(TotalAmount) OVER (
        ORDER BY OrderDate
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
    ) as MovingAvg3,

    -- Year-to-date total
    SUM(TotalAmount) OVER (
        PARTITION BY YEAR(OrderDate)
        ORDER BY OrderDate
    ) as YearToDateTotal,

    -- Previous order amount
    LAG(TotalAmount, 1) OVER (ORDER BY OrderDate) as PrevOrderAmount,

    -- Percentage of monthly total
    TotalAmount * 100.0 / SUM(TotalAmount) OVER (
        PARTITION BY YEAR(OrderDate), MONTH(OrderDate)
    ) as PercentOfMonthlyTotal
FROM Orders
ORDER BY OrderDate;
```

Q26: Find top N records per group.

Answer:

```
-- Top 3 highest paid employees per department
WITH RankedEmployees AS (
    SELECT
        EmployeeID,
        EmployeeName,
        DepartmentID,
        Salary,
        ROW_NUMBER() OVER (PARTITION BY DepartmentID ORDER BY Salary DESC) as RowNum
    FROM Employees
)
SELECT * FROM RankedEmployees WHERE RowNum &lt;= 3;

-- Alternative using correlated subquery
SELECT e1.*
FROM Employees e1
WHERE 3 &gt;= (
    SELECT COUNT(*)
    FROM Employees e2
```

```
    WHERE e2.DepartmentID = e1.DepartmentID
    AND e2.Salary &gt;= e1.Salary
)
ORDER BY DepartmentID, Salary DESC;
```

Database Administration

Q27: What are transactions and ACID properties?

Answer:

Transaction: Sequence of database operations treated as single unit of work.

ACID Properties:

Atomicity:

- All operations succeed or all fail
- No partial transactions

Consistency:

- Database remains in valid state
- All constraints maintained

Isolation:

- Concurrent transactions don't interfere
- Each transaction sees consistent view

Durability:

- Committed changes persist
- Survive system failures

Transaction Control:

```
-- Start transaction
BEGIN TRANSACTION;

-- Perform operations
UPDATE Accounts SET Balance = Balance - 100 WHERE AccountID = 1;
UPDATE Accounts SET Balance = Balance + 100 WHERE AccountID = 2;

-- Check for errors and commit or rollback
IF @@ERROR = 0
    COMMIT TRANSACTION;
ELSE
    ROLLBACK TRANSACTION;
```

Q28: Explain isolation levels and their effects.

Answer:

Isolation Levels (from least to most restrictive):

READ UNCOMMITTED:

- Can read uncommitted changes
- Dirty reads possible
- Lowest isolation, highest performance

READ COMMITTED (Default):

- Only reads committed data
- Prevents dirty reads
- Non-repeatable reads possible

REPEATABLE READ:

- Same data returned on multiple reads
- Prevents dirty and non-repeatable reads
- Phantom reads possible

SERIALIZABLE:

- Highest isolation level
- Prevents all phenomena
- Lowest performance

Setting Isolation Level:

```
-- Set isolation level for session
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

-- Set for specific transaction
BEGIN TRANSACTION
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
-- ... transaction operations
COMMIT;
```

Real-World Scenarios

Q29: Design a query to find customers with declining purchase patterns.

Answer:

```
WITH MonthlyPurchases AS (
    SELECT
        CustomerID,
        YEAR(OrderDate) as OrderYear,
        MONTH(OrderDate) as OrderMonth,
        COUNT(*) as OrderCount,
        SUM(TotalAmount) as MonthlySpend
    FROM Orders
    WHERE OrderDate &gt;= DATEADD(month, -6, GETDATE())  -- Last 6 months
    GROUP BY CustomerID, YEAR(OrderDate), MONTH(OrderDate)
),
PurchaseTrends AS (
    SELECT
        CustomerID,
        OrderYear,
        OrderMonth,
        MonthlySpend,
        LAG(MonthlySpend, 1) OVER (
            PARTITION BY CustomerID
            ORDER BY OrderYear, OrderMonth
        ) as PrevMonthSpend,
        LAG(MonthlySpend, 2) OVER (
            PARTITION BY CustomerID
            ORDER BY OrderYear, OrderMonth
        ) as TwoMonthsAgoSpend
    FROM MonthlyPurchases
)
SELECT DISTINCT
    pt.CustomerID,
    c.CustomerName,
    pt.MonthlySpend as CurrentSpend,
    pt.PrevMonthSpend,
    pt.TwoMonthsAgoSpend
FROM PurchaseTrends pt
JOIN Customers c ON pt.CustomerID = c.CustomerID
WHERE pt.MonthlySpend &lt; pt.PrevMonthSpend
  AND pt.PrevMonthSpend &lt; pt.TwoMonthsAgoSpend
  AND pt.TwoMonthsAgoSpend IS NOT NULL;
```

Q30: Create a query for inventory management with reorder alerts.

Answer:

```
WITH InventoryStatus AS (
    SELECT
        p.ProductID,
        p.ProductName,
        p.Category,
        p.ReorderLevel,
        p.MaxStock,
```

```
        COALESCE(inv.CurrentStock, 0) as CurrentStock,

        -- Calculate average daily sales (last 30 days)
        COALESCE(
            (SELECT AVG(daily_sales)
             FROM (
                 SELECT DATE(o.OrderDate) as sale_date, SUM(oi.Quantity) as daily_sales
                 FROM Orders o
                 JOIN OrderItems oi ON o.OrderID = oi.OrderID
                 WHERE oi.ProductID = p.ProductID
                   AND o.OrderDate &gt;= DATEADD(day, -30, GETDATE())
                 GROUP BY DATE(o.OrderDate)
             ) daily_totals), 0
        ) as AvgDailySales,

        -- Calculate days of stock remaining
        CASE
            WHEN COALESCE(
                (SELECT AVG(daily_sales)
                 FROM (
                     SELECT DATE(o.OrderDate) as sale_date, SUM(oi.Quantity) as daily_sal
                     FROM Orders o
                     JOIN OrderItems oi ON o.OrderID = oi.OrderID
                     WHERE oi.ProductID = p.ProductID
                       AND o.OrderDate &gt;= DATEADD(day, -30, GETDATE())
                     GROUP BY DATE(o.OrderDate)
                 ) daily_totals), 0
            ) &gt; 0
            THEN COALESCE(inv.CurrentStock, 0) /
                 COALESCE(
                     (SELECT AVG(daily_sales)
                      FROM (
                          SELECT DATE(o.OrderDate) as sale_date, SUM(oi.Quantity) as dail
                          FROM Orders o
                          JOIN OrderItems oi ON o.OrderID = oi.OrderID
                          WHERE oi.ProductID = p.ProductID
                            AND o.OrderDate &gt;= DATEADD(day, -30, GETDATE())
                          GROUP BY DATE(o.OrderDate)
                      ) daily_totals), 1
                 )
            ELSE 999
        END as DaysOfStockRemaining

    FROM Products p
    LEFT JOIN Inventory inv ON p.ProductID = inv.ProductID
)
SELECT
    ProductID,
    ProductName,
    Category,
    CurrentStock,
    ReorderLevel,
    MaxStock,
    AvgDailySales,
    ROUND(DaysOfStockRemaining, 1) as DaysOfStockRemaining,
```

```
    -- Reorder recommendations
    CASE
        WHEN CurrentStock &lt;= ReorderLevel THEN 'URGENT: Reorder Now'
        WHEN DaysOfStockRemaining &lt;= 7 THEN 'WARNING: Reorder Soon'
        WHEN DaysOfStockRemaining &lt;= 14 THEN 'CAUTION: Monitor Stock'
        ELSE 'OK'
    END as StockStatus,

    -- Suggested order quantity
    GREATEST(MaxStock - CurrentStock, 0) as SuggestedOrderQty

FROM InventoryStatus
ORDER BY
    CASE
        WHEN CurrentStock &lt;= ReorderLevel THEN 1
        WHEN DaysOfStockRemaining &lt;= 7 THEN 2
        WHEN DaysOfStockRemaining &lt;= 14 THEN 3
        ELSE 4
    END,
    DaysOfStockRemaining;
```

Quick Reference Guide

SQL Execution Order

1. **FROM** (including JOINs)
2. **WHERE**
3. **GROUP BY**
4. **HAVING**
5. **SELECT**
6. **ORDER BY**
7. **LIMIT/TOP**

Common Functions

```
-- String Functions
UPPER(string), LOWER(string), LENGTH(string)
SUBSTRING(string, start, length)
REPLACE(string, old, new)
CONCAT(string1, string2)

-- Date Functions
GETDATE(), CURRENT_DATE, NOW()
DATEADD(interval, number, date)
DATEDIFF(interval, date1, date2)
YEAR(date), MONTH(date), DAY(date)

-- Conditional Functions
CASE WHEN condition THEN result ELSE default END
```

```
COALESCE(value1, value2, ...)
NULLIF(value1, value2)
```

Performance Tips

1. **Use indexes** on frequently queried columns
2. **Avoid SELECT *** - specify needed columns
3. **Use LIMIT** for large result sets
4. **Use EXISTS** instead of IN for subqueries
5. **Avoid functions** in WHERE clauses
6. **Use appropriate JOIN types**
7. **Consider query execution plan**

Interview Tips

1. **Understand the business context** of the question
2. **Ask clarifying questions** about requirements
3. **Start with basic solution**, then optimize
4. **Explain your thought process** out loud
5. **Test edge cases** (NULL values, empty results)
6. **Know multiple approaches** to solve problems
7. **Practice on real datasets** before interviews

This comprehensive SQL cheat sheet covers the most important concepts tested in data science and software engineering interviews. Practice these concepts with real databases to build confidence and expertise.